

DNNKit: A Lightweight Framework for Reproducible Deep Learning Experiments

Festus M. Slade

Cambridge Neuroscience, University of Cambridge, Cambridge CB2 1TN, UK
github.com/Festus0/dnnkit

March 10, 2026

Abstract

Reproducibility and experiment management remain major challenges in modern deep learning research. While powerful frameworks such as PyTorch Lightning and HuggingFace Trainer provide extensive tooling, their complexity can introduce significant overhead for small-scale research projects, educational settings, and rapid experimentation workflows. We present **DNNKit**, a lightweight framework for reproducible deep learning experimentation that integrates a command-line training interface, dataset registry, experiment logging, benchmark tracking, and automatic report generation. DNNKit aims to provide a minimal, transparent infrastructure that simplifies experiment management while maintaining compatibility with PyTorch. We demonstrate the functionality of the framework using a benchmark experiment on the MNIST handwritten digit classification dataset.

1 Introduction

Deep learning research often involves large numbers of experiments, parameter configurations, datasets, and model variants. Managing these experiments in a reproducible and structured way is a persistent challenge.

Common research workflows frequently rely on ad-hoc scripts, notebooks, and manually maintained logs. This can make it difficult to reproduce results, compare experiments, or maintain clear experiment provenance.

Several frameworks have been developed to address these issues. For example, PyTorch Lightning provides a structured training interface, while HuggingFace Trainer focuses on large-scale model training and benchmarking. Although these frameworks are powerful, they introduce additional layers of abstraction that may be unnecessary for smaller research projects or educational environments.

In this work, we introduce **DNNKit**, a lightweight experimentation framework designed to simplify experiment management while remaining transparent

and easy to modify. The framework focuses on four core components:

- Command-line training interface
- Dataset registry
- Experiment logging and result storage
- Automatic leaderboard and report generation

DNNKit is designed to provide a minimal infrastructure for reproducible experiments while remaining fully compatible with PyTorch-based models.

2 Framework Design

The design philosophy of DNNKit prioritizes simplicity, transparency, and reproducibility.

2.1 Training CLI

The framework provides a command-line interface (CLI) that standardizes experiment execution. Training runs can be launched using a simple command:

```
dnnkit train --dataset mnist --epochs 3
```

Each training run automatically generates a dedicated experiment directory containing:

- model checkpoints
- training metrics
- configuration parameters

2.2 Dataset Registry

Datasets are accessed through a registry mechanism that standardizes data loading. This avoids repeated boilerplate code across experiments and ensures consistent dataset usage.

2.3 Experiment Logging

All training runs are automatically logged to structured output directories. Metrics such as training loss, accuracy, and configuration parameters are stored in machine-readable formats.

2.4 Leaderboard Tracking

DNNKit aggregates experiment results into a global leaderboard, allowing quick comparison across runs.

2.5 Automatic Report Generation

The framework can automatically generate experiment reports summarizing results and benchmark performance.

3 Experimental Evaluation

To demonstrate the functionality of DNNKit, we evaluated the framework using the MNIST handwritten digit classification dataset.

3.1 Dataset

MNIST consists of 70,000 grayscale images of handwritten digits with a resolution of 28×28 pixels.

3.2 Model

We used a simple fully connected neural network consisting of:

- Input layer (784 features)
- Hidden layer with ReLU activation
- Output layer with 10 classes

The model was trained using the Adam optimizer.

3.3 Training Configuration

- Optimizer: Adam
- Learning rate: 1×10^{-3}
- Batch size: 64
- Epochs: 3

4 Results

Table 1 summarizes the benchmark results obtained using DNNKit.

The results demonstrate that the framework supports standard deep learning training workflows while maintaining a lightweight and transparent architecture.

Epoch	Training Loss	Test Accuracy
1	0.3442	94.46%
2	0.1610	95.95%
3	0.1110	96.97%

Table 1: MNIST benchmark results obtained using DNNKit.

5 Discussion

DNNKit aims to fill the gap between simple research scripts and large training frameworks.

Compared to large frameworks, DNNKit offers several advantages:

- minimal abstraction overhead
- easy code inspection
- fast experiment setup
- reproducible experiment tracking

The framework is particularly well suited for:

- rapid research prototyping
- small-scale machine learning experiments
- educational environments

Future development may include additional features such as distributed training support, extended dataset registries, and experiment visualization tools.

6 Conclusion

We introduced DNNKit, a lightweight framework for reproducible deep learning experiments. The framework integrates experiment management, dataset handling, benchmarking, and report generation into a simple command-line interface.

Through a benchmark experiment on MNIST, we demonstrated that DNNKit provides a practical infrastructure for organizing and reproducing machine learning experiments.

The source code is publicly available at:

<https://github.com/Festus0/dnnkit>

Acknowledgments

This work was developed as part of independent research into lightweight machine learning infrastructure for reproducible experimentation.

References

- [1] Paszke, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS (2019).
- [2] LeCun, Y. et al. Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE (1998).
- [3] Falcon, W. PyTorch Lightning. GitHub Repository.